

Dynamic Analysis Android Application

Day2 : Lab manual

Table of Content

1. Objectives	4
3. Setup Tools for Windows	4
3.1 Frida Client and Server Setup	4
3.1.1 Verify Python	4
3.1.2 Install Frida Client Tools	5
3.1.3 Verify Frida Client Version	5
3.1.4 Verify Emulator Connection	5
4. Setup Tools for Linux (Ubuntu)	7
4.1 Frida Client and Server Setup	7
4.1.1 Verify Python and Pip	7
4.1.2 Install Frida Client Tools	7
4.1.3 Verify the Frida Client Version	8
4.1.4 Verify ADB Connection	8
5. Dynamic Analysis for Windows and Linux	9
5.1 ADB Dynamic Analysis for Windows and Linux	9
5.1.1 Confirming ADB Connection to Device	9
5.2 Collect Emulator Information	10
5.3 Install the Target APK	12
5.4 Check Installed APK Path	14
5.5 Collect Package Information	15
5.6 Check Whether the Application Is Running	19
5.7 Start and Stop the Application Using ADB	21
5.8 Application Storage Analysis	22
5.8.1 Check run-as Access	22
5.8.2 List Application Sandbox Files	23
5.8.3 Root-Capable Emulator Method	25
5.9 Send a Known Intent to an Authorized Component	26
Start a Known Activity	26
6. ADB Logcat Analysis for Windows and Linux	27
6.1 Clear Existing Logcat Data	27
6.2 Start Full Logcat Capture	27
6.3 Capture Logs for One Application Process	28
6.4 Filter Logcat Using the Package Name	29
6.5 Filter Logs by Severity	30
6.6 Search Saved Logs for Sensitive Information	31

7.Frida analysis	32
7.1 Verify Emulator and ADB Connection	32
7.2 Identify Emulator Architecture	33
7.3 Download and Extract Matching Frida Server	34
7.3.1 Open the Official Frida Releases Page	34
7.3.2 Select the Exact Matching Server File	34
7.3.3 Extract the .xz Archive	35
7.4 Push and Start Frida Server	37
7.4.1 Push Frida Server to the Emulator	37
7.4.2 Open an ADB Shell and Obtain Root Access	37
7.4.4 Give Execute Permission to Frida Server	38
7.4.5 Start Frida Server in the Background	38
7.4.6 Confirm that Frida Server is Running	39
7.5 Verify Frida Connection	40
7.6 Spawn the Target Application with Frida	40
7.6.1 Identify the Target Package Name	40
7.6.2 Spawn the Target Application	41
7.6.3 Exit the Frida Console	42
7.7 Basic Frida Script Test	42
8. Identify Security Findings and Prepare Observations	44

1. Objectives

The objective of this lab is to install and configure different tools used in Dynamic Analysis of the application for Windows Operating System and Linux.

2. Prerequisites

Requirement	Details
Operating System	Windows 10/11 (64-bit) or Ubuntu Linux 20.04/22.04/24.04
Hardware	• CPU: x86_64 CPU architecture • RAM: 8 GB RAM or more • Disk Space: 10 GB of available disk space minimum (IDE + Android SDK + Android Emulator)
Tools Required	Android Studio, AVD Emulator, Jadx, Frida, adb

3. Setup Tools for Windows

Note: All required tools are installed in the previous lab except Frida.

3.1 Frida Client and Server Setup

3.1.1 Verify Python

Open **PowerShell** and run:

```
Shell
python --version
python -m pip --version
```

3.1.2 Install Frida Client Tools

Shell

```
python -m pip install --upgrade frida-tools
```

```

C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> python --version
Python 3.12.8

C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> python -m pip --version
pip 24.3.1 from C:\Users\vinay\AppData\Local\Programs\Python\Python312\Lib\site-packages\pip (python 3.12.8)

C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> python -m pip install --upgrade frida-tools
Collecting frida-tools
  Downloading frida_tools-14.18.4.tar.gz (4.7 MB)
    4.7/4.7 MB 11.8 MB/s eta 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Collecting colorama<1.0.0,>=0.2.7 (from frida-tools)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting frida<18.0.0,>=17.10.0 (from frida-tools)
  Downloading frida-17.18.0-cp37-abi3-win_and64.whl.metadata (2.3 kB)
Collecting prompt-toolkit<4.0.0,>=2.0.0 (from frida-tools)
  
```

3.1.3 Verify Frida Client Version

Shell

```
frida --version
```

Record the exact **version**. It must be the same version as the **Android Frida server**.

```

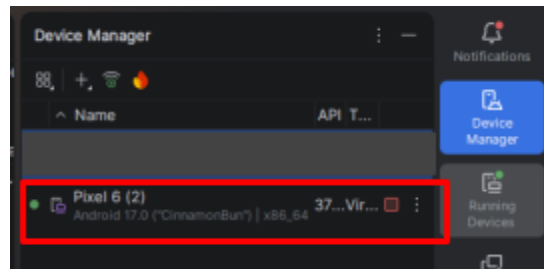
C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> frida --version
17.18.0
C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> |

```

3.1.4 Verify Emulator Connection

Start the **Android emulator** and run (which is not having the **playstore** symbol):

1. Open **android studio**
2. Click on **device manager**
3. Click on the **play button**



4. Run the **emulator** (configured in previous lab)



```
Shell
adb devices
```

Expected result:

```
None
List of devices attached
emulator-5554    device
```

4. Setup Tools for Linux (Ubuntu)

4.1 Frida Client and Server Setup

4.1.1 Verify Python and Pip

Open **Terminal** and run:

```
Shell  
python3 --version  
python3 -m pip --version
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ python --version  
python -m pip --version  
Python 3.10.12  
pip 26.2.1 from /home/cdac/.local/lib/python3.10/site-packages/pip (python 3.10)
```

If **pip** is not installed:

```
Shell  
sudo apt update  
sudo apt install -y python3-pip python3-venv
```

4.1.2 Install Frida Client Tools

```
Shell  
python3 -m pip install --upgrade frida-tools
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ python -m pip install --upgrade frida-tools
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: frida-tools in ./local/lib/python3.10/site-packages (14.10.4)
Requirement already satisfied: colorama<1.0.0,>=0.2.7 in ./local/lib/python3.10/site-packages (from frida-tools) (0.4.5)
Requirement already satisfied: frida<18.0.0,>=17.10.0 in ./local/lib/python3.10/site-packages (from frida-tools) (17.18.0)
Requirement already satisfied: prompt-toolkit<4.0.0,>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from frida-tools) (3.0.51)
```

If Ubuntu prevents system-wide **pip** installation, use a **virtual environment**:

```
Shell
python3 -m venv ~/vapt-frida-env
source ~/vapt-frida-env/bin/activate
python -m pip install --upgrade pip frida-tools
```

4.1.3 Verify the Frida Client Version

Run:

```
Shell
frida --version
```

The lab evidence shows this **output**:

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ frida --version
17.18.0
```

4.1.4 Verify ADB Connection

Start the **Android emulator** and run:

```
Shell
adb devices
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ adb devices
List of devices attached
emulator-5554    device
```

Expected result:

None

```
List of devices attached
emulator-5554    device
```

Continue with the common dynamic-analysis procedure in Section 5.

5. Dynamic Analysis for Windows and Linux

Note: The remaining Frida deployment and dynamic-analysis commands are the same on Windows PowerShell and Ubuntu Terminal. Ensure that **adb** and **frida** are available in PATH before proceeding.

5.1 ADB Dynamic Analysis for Windows and Linux

Note: ADB installation and **emulator** setup are already completed. Start the Android emulator before beginning the practical exercises. Use only the instructor-provided vulnerable application, such as **AndroGoat** or **VulDroid**, and dummy test data.

5.1.1 Confirming ADB Connection to Device

Open **PowerShell** on Windows or **Terminal** on **Linux** and run:

Shell

```
adb devices
```

Expected output:

None

```
List of devices attached
emulator-5554    device
```

```
PS C:\Users\vinav\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool
> adb devices
List of devices attached
emulator-5554    device
```

Output	Meaning
emulator-5554 device	Emulator is connected and ready
offline	Wait for emulator boot or restart ADB
unauthorized	Accept the USB-debugging prompt on the device if applicable
No devices listed	Start the emulator and run the command again

If the device is not listed, run:

```
Shell
adb kill-server
adb start-server
adb devices
```

5.2 Collect Device Information

Run the following commands and record the **output** in the lab report:

```
Shell
adb shell getprop ro.product.model
adb shell getprop ro.build.version.release
adb shell getprop ro.build.version.sdk
adb shell getprop ro.product.cpu.abi
```

Linux :

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ adb shell getprop ro.product.model
adb shell getprop ro.build.version.release
adb shell getprop ro.build.version.sdk
adb shell getprop ro.product.cpu.abi
sdk_gphone16k_x86_64
17
37
```

Windows powershell:

```

PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell getprop ro.build.ve
rsion.release
17
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell getprop ro.build.ve
rsion.sdk
37
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell getprop ro.product.
cpu.abi
x86_64
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> |
  
```

Example output:

```

None
Android SDK built for x86_64
15
35
x86_64
  
```

Command	Analysis Purpose
adb shell getprop ro.product.model	Identifies the emulator/device model
adb shell getprop ro.build.version.release	Shows Android version
adb shell getprop ro.build.version.sdk	Shows Android API level
adb shell getprop ro.product.cpu.abi	Shows device CPU architecture, useful for Frida server selection

5.3 Install the Target APK

Before identifying the package name, install the target **APK** on the **emulator**.

Assume the APK is stored at:

Path (in your system)

~/apks/vuldroid.apk

Install it with:

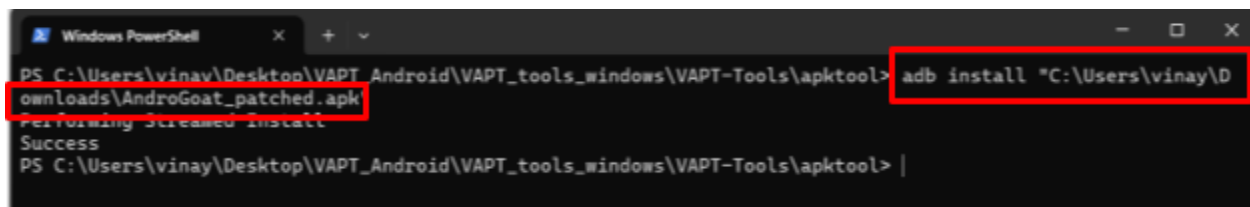
Bash

adb install ~/apks/vuldroid.apk

On **Windows PowerShell**:

powershell

adb install "C:\Users\<USERNAME>\Downloads\vuldroid.apk"



```
Windows PowerShell
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb install "C:\Users\vinay\Downloads\AndroGoat_patched.apk"
Performing Streamed Install
Success
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> |
```

Expected successful output:

text

Performing Streamed Install

Success

Reinstall during testing

If the application is already **installed** and you want to **replace** it while retaining its data:

bash

adb install -r ~/apks/vuldroid.apk

Verify the installation

You can confirm that the **APK** was installed by **listing packages**:

bash

`adb shell pm list packages | grep -i vuldroid`

```
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell pm list packages |
Select-String -Pattern "goat"
package:owasp.sat.goat
```

5.3.1 Identify the Target **Application Package**

Do not assume the package name. Identify it first.

For **VulDroid**, run:

Linux

Shell

`adb shell pm list packages | grep vuldroid`

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ adb shell pm list packages | grep vuldroid
package:com.vuldroid.application
```

Expected output:

None

`package:com.vuldroid.application`

For **AndroGoat**, run:

Shell

`adb shell pm list packages | grep -i goat`

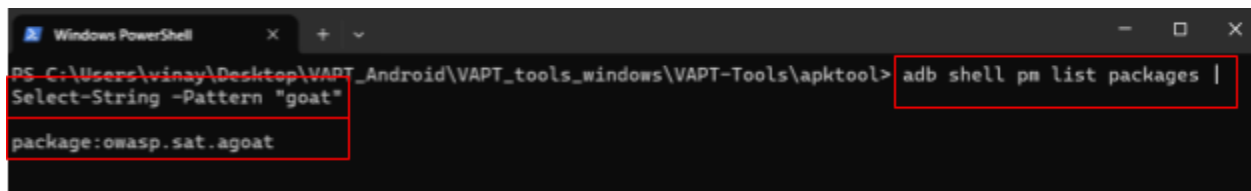
On **Windows PowerShell**, use:

```
Shell  
adb shell pm list packages | Select-String -Pattern "vuldroid"
```

Or:

```
Shell  
adb shell pm list packages | Select-String -Pattern "goat"
```

In the remaining examples, replace <TARGET_PACKAGE> with the actual application package name (e.g., com.vuldroid.application).



```
Windows PowerShell  
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell pm list packages |  
Select-String -Pattern "goat"  
package:owasp.sat.agoat
```

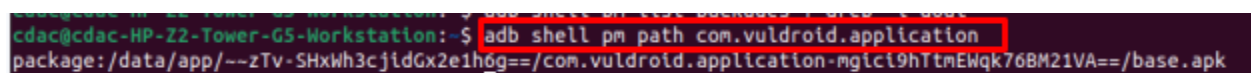
5.4 Check Installed APK Path

Run:

```
Shell  
adb shell pm path <TARGET_PACKAGE>
```

Example:

```
Shell  
adb shell pm path com.vuldroid.application
```



```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ adb shell pm path com.vuldroid.application  
package: /data/app/...zTv-SHxWh3cjidGx2e1h0g==/com.vuldroid.application-mgic19hTtmEWqk76BM21VA==/base.apk
```

Windows powershell :

```
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell pm path owasp.sat.a
goat
package:/data/app/~~jPGtRVJI_6JSfoPmlUKVdw==/owasp.sat.agoat-0xi_WboekdQGmPUjuvRZtA==/base.apk
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> |
```

Expected output:

None

```
package:/data/app/~~<random-value>/com.vuldroid.application-<random
-value>/base.apk
```

This command confirms that the application is installed and shows the **APK** installation **location** on the emulator.

5.5 Collect Package Information

Run:

Shell

```
adb shell dumpsys package <TARGET_PACKAGE>
```

Example:

Shell

```
adb shell dumpsys package com.vuldroid.application
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ adb shell dumpsys package com.vuldroid.application
Activity Resolver Table:
  Schemes:
    http:
      2572cf5 com.vuldroid.application/.BlogsViewer filter a8c06eb
        Action: "android.intent.action.VIEW"
        Category: "android.intent.category.DEFAULT"
        Category: "android.intent.category.BROWSABLE"
        Scheme: "http"
        Scheme: "https"
        Authority: "medium.com": -1
        Authority: "medium.com": -1
    https:
      2572cf5 com.vuldroid.application/.BlogsViewer filter a8c06eb
        Action: "android.intent.action.VIEW"
        Category: "android.intent.category.DEFAULT"
        Category: "android.intent.category.BROWSABLE"
        Scheme: "http"
        Scheme: "https"
        Authority: "medium.com": -1
        Authority: "medium.com": -1
```

This produces detailed package information. Focus on the following items: versionName

- versionCode
- debuggable
- requested permissions
- activities
- services
- receivers
- providers

Filter Important Information

Linux:

Shell

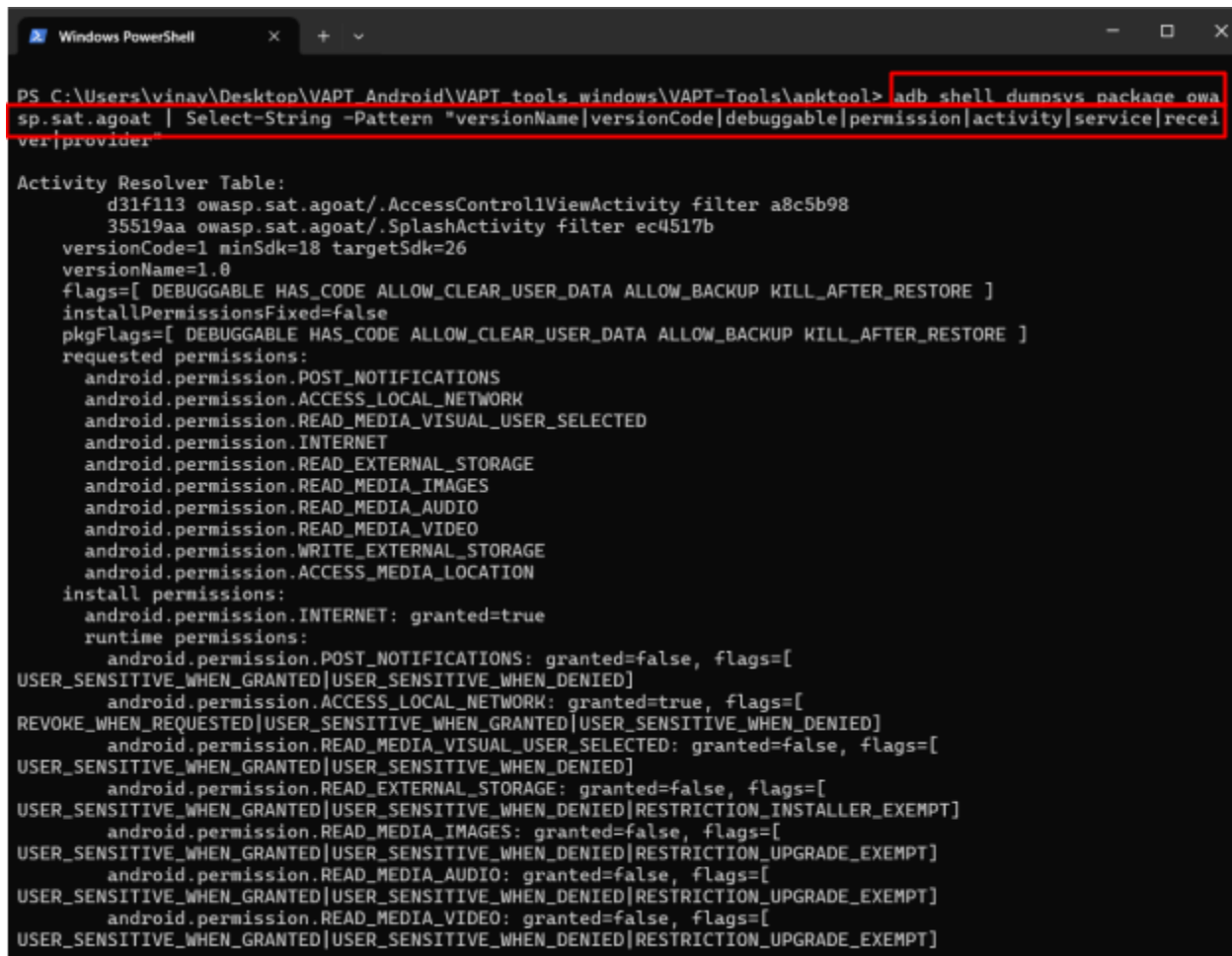
```
adb shell dumpsys package <TARGET_PACKAGE> | grep -Ei
"versionName|versionCode|debuggable|permission|activity|service|rec
eiver|provider"
```

```
cdac@cdac-HP-Z2-Tower-GS-Workstation:~$ adb shell dumpsys package com.vulldroid.application
| grep -Ei "versionName|versionCode|debuggable|permission|activity|service|receiver|provider"
ActivityManager: Tables:
Schemes:
  http:
    2572cf5 com.vulldroid.application/.BlogsViewer filter a8c06eb
      Action: "android.intent.action.VIEW"
      Category: "android.intent.category.DEFAULT"
      Category: "android.intent.category.BROWSABLE"
      Scheme: "http"
      Scheme: "https"
      Authority: "medlun.com": -1
      Authority: "medlun.com": -1
  https:
    2572cf5 com.vulldroid.application/.BlogsViewer filter a8c06eb
      Action: "android.intent.action.VIEW"
      Category: "android.intent.category.DEFAULT"
      Category: "android.intent.category.BROWSABLE"
      Scheme: "http"
      Scheme: "https"
      Authority: "medlun.com": -1
      Authority: "medlun.com": -1
    74bdd18 com.vulldroid.application/.ForgetPassword filter a2a4ae1
      Action: "android.intent.action.VIEW"
      Category: "android.intent.category.DEFAULT"
      Category: "android.intent.category.BROWSABLE"
      Scheme: "https"
      Authority: ".firebaseapp.com": -1 WILD
```

Windows PowerShell:

Shell

```
adb shell dumpsys package <TARGET_PACKAGE> | Select-String -Pattern
"versionName|versionCode|debuggable|permission|activity|service|receiver|provider"
```



```
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell dumpsys package owa
sp.sat.agoat | Select-String -Pattern "versionName|versionCode|debuggable|permission|activity|service|receiver|provider"

Activity Resolver Table:
  d31f113 owasp.sat.agoat/.AccessControlViewActivity filter a8c5b98
  35519aa owasp.sat.agoat/.SplashActivity filter ec4517b
  versionCode=1 minSdk=18 targetSdk=26
  versionName=1.0
  flags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP KILL_AFTER_RESTORE ]
  installPermissionsFixed=false
  pkgFlags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP KILL_AFTER_RESTORE ]
  requested permissions:
    android.permission.POST_NOTIFICATIONS
    android.permission.ACCESS_LOCAL_NETWORK
    android.permission.READ_MEDIA_VISUAL_USER_SELECTED
    android.permission.INTERNET
    android.permission.READ_EXTERNAL_STORAGE
    android.permission.READ_MEDIA_IMAGES
    android.permission.READ_MEDIA_AUDIO
    android.permission.READ_MEDIA_VIDEO
    android.permission.WRITE_EXTERNAL_STORAGE
    android.permission.ACCESS_MEDIA_LOCATION
  install permissions:
    android.permission.INTERNET: granted=true
  runtime permissions:
    android.permission.POST_NOTIFICATIONS: granted=false, flags=[
    USER_SENSITIVE_WHEN_GRANTED|USER_SENSITIVE_WHEN_DENIED]
    android.permission.ACCESS_LOCAL_NETWORK: granted=true, flags=[
    REVOKE_WHEN_REQUESTED|USER_SENSITIVE_WHEN_GRANTED|USER_SENSITIVE_WHEN_DENIED]
    android.permission.READ_MEDIA_VISUAL_USER_SELECTED: granted=false, flags=[
    USER_SENSITIVE_WHEN_GRANTED|USER_SENSITIVE_WHEN_DENIED]
    android.permission.READ_EXTERNAL_STORAGE: granted=false, flags=[
    USER_SENSITIVE_WHEN_GRANTED|USER_SENSITIVE_WHEN_DENIED|RESTRICTION_INSTALLER_EXEMPT]
    android.permission.READ_MEDIA_IMAGES: granted=false, flags=[
    USER_SENSITIVE_WHEN_GRANTED|USER_SENSITIVE_WHEN_DENIED|RESTRICTION_UPGRADE_EXEMPT]
    android.permission.READ_MEDIA_AUDIO: granted=false, flags=[
    USER_SENSITIVE_WHEN_GRANTED|USER_SENSITIVE_WHEN_DENIED|RESTRICTION_UPGRADE_EXEMPT]
    android.permission.READ_MEDIA_VIDEO: granted=false, flags=[
    USER_SENSITIVE_WHEN_GRANTED|USER_SENSITIVE_WHEN_DENIED|RESTRICTION_UPGRADE_EXEMPT]
```

```
Windows PowerShell
extractNativeLibs=true
primaryCpuAbi=null
secondaryCpuAbi=null
cpuAbiOverride=null
versionCode=1 minSdk=18 targetSdk=26
minExtensionVersions=[]
versionName=1.0
hiddenApiEnforcementPolicy=2
usesNonSdkApi=false
splits=[base]
apkSigningVersion=3
flags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP KILL_AFTER_RESTORE ]
privateFlags=[ PRIVATE_FLAG_ACTIVITIES_RESIZE_MODE_RESIZEABLE_VIA_SDK_VERSION PRIVATE_FLAG_REQUEST_LEGACY_EXTERNAL_STORAGE PRIVATE_FLAG_ALLOW_NATIVE_HEAP_POINTER_TAGGING ]
forceQueryable=false
pageSizeCompat=0
scannedAsStoppedSystemApp=false
supportsScreens=[small, medium, large, xlarge, resizeable, anyDensity]
usesLibraries:
```

```
Windows PowerShell
hiddenApiEnforcementPolicy=2
usesNonSdkApi=false
splits=[base]
apkSigningVersion=3
flags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP KILL_AFTER_RESTORE ]
privateFlags=[ PRIVATE_FLAG_ACTIVITIES_RESIZE_MODE_RESIZEABLE_VIA_SDK_VERSION PRIVATE_FLAG_REQUEST_LEGACY_EXTERNAL_STORAGE PRIVATE_FLAG_ALLOW_NATIVE_HEAP_POINTER_TAGGING ]
forceQueryable=false
pageSizeCompat=0
scannedAsStoppedSystemApp=false
supportsScreens=[small, medium, large, xlarge, resizeable, anyDensity]
usesLibraries:
  android.test.base
  org.apache.http.legacy
```

```
Windows PowerShell
pkgFlags=[ DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP KILL_AFTER_RESTORE ]
privatePkgFlags=[ PRIVATE_FLAG_ACTIVITIES_RESIZE_MODE_RESIZEABLE_VIA_SDK_VERSION PRIVATE_FLAG_REQUEST_LEGACY_EXTERNAL_STORAGE PRIVATE_FLAG_ALLOW_NATIVE_HEAP_POINTER_TAGGING ]
apexModuleName=null
requested permissions:
  android.permission.POST_NOTIFICATIONS
  android.permission.ACCESS_LOCAL_NETWORK
  android.permission.READ_MEDIA_VISUAL_USER_SELECTED
  android.permission.INTERNET
  android.permission.READ_EXTERNAL_STORAGE
  android.permission.READ_MEDIA_IMAGES
  android.permission.READ_MEDIA_AUDIO
  android.permission.READ_MEDIA_VIDEO
  android.permission.WRITE_EXTERNAL_STORAGE
  android.permission.ACCESS_MEDIA_LOCATION
install permissions:
  android.permission.INTERNET: granted=true
usesPermissions=[android.permission.ACCESS_NETWORK_STATE, android.permission.ACCESS_WIFI_STATE, android.permission.INTERNET, android.permission.READ_EXTERNAL_STORAGE, android.permission.WRITE_EXTERNAL_STORAGE, android.permission.ACCESS_MEDIA_LOCATION, android.permission.REQUEST_INSTALL_PACKAGES]
```

What to Analyze

Item	What to Check
versionName and versionCode	Record the exact application version tested
debuggable	A release application should normally not be debuggable
Permissions	Check whether sensitive permissions are present
Activities	Identify activities that may be accessible externally
Services	Review background services and their exposure
Receivers	Review broadcast receivers and intent handling
Providers	Review content providers and data exposure

5.6 Check Whether the Application Is Running

Open the target application on the emulator. Then run:

```
Shell
adb shell pidof <TARGET_PACKAGE>
```

Example:

```
Shell
adb shell pidof com.vulldroid.application
```

Expected output:

```
None
12345
```

The number is the **Process ID**, or **PID**.

```

Windows PowerShell
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell pidof owasp.sat.ago
at
22544
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool>
  
```

Linux :

```

cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ adb shell pidof com.vuldroid.application
4056
  
```

You can also list the process:

Linux:

```

Shell
adb shell ps -A | grep <TARGET_PACKAGE>
  
```

Windows PowerShell:

```

Shell
adb shell ps -A | Select-String -Pattern "<TARGET_PACKAGE>"
  
```

Example:

```

Shell
adb shell ps -A | grep com.vuldroid.application
  
```

linux:

```

cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ adb shell ps -A | grep com.vuldroid.application
u0_a229      4056   471   16378212  50168 0          0 S com.vuldroid.application
  
```

```

Windows PowerShell
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell ps -A | Select-String -Pattern owasp.sat.agoat
u0_a230      22544  483   14436308  43916 0          0 S owasp.sat.agoat
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool>
  
```

5.7 Start and Stop the Application Using ADB

Start the Main Application Activity (to launch the application)

```
Shell
adb shell monkey -p <TARGET_PACKAGE> 1
```

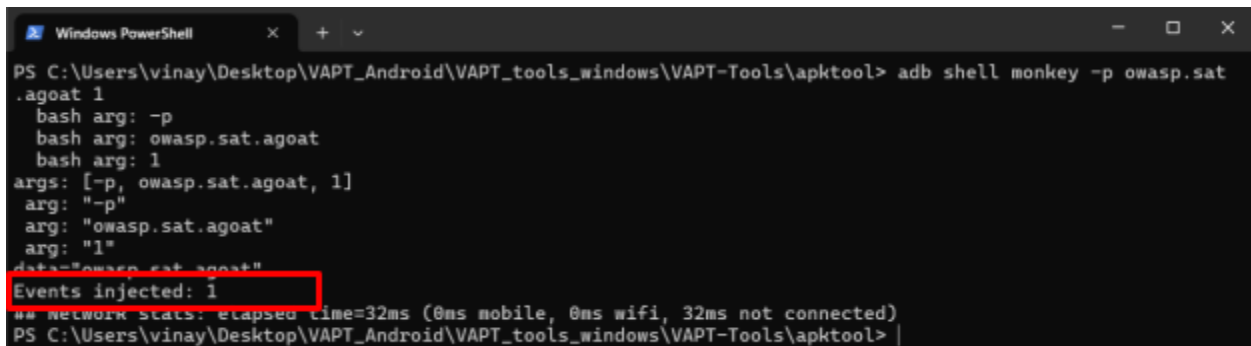
Example:

```
Shell
adb shell monkey -p com.vuldroid.application 1
```

Expected output resembles:

```
None
Events injected: 1
```

This starts the application's launcher activity.



```
Windows PowerShell
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell monkey -p owasp.sat
.agoat 1
  bash arg: -p
  bash arg: owasp.sat.agoat
  bash arg: 1
args: [-p, owasp.sat.agoat, 1]
arg: "-p"
arg: "owasp.sat.agoat"
arg: "1"
data="owasp.sat.agoat"
Events injected: 1
** network stats: elapsed time=32ms (0ms mobile, 0ms wifi, 32ms not connected)
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> |
```

Linux :

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ adb shell monkey -p com.vuldroid.application 1
  bash arg: -p
  bash arg: com.vuldroid.application
  bash arg: 1
args: [-p, com.vuldroid.application, 1]
arg: "-p"
arg: "com.vuldroid.application"
arg: "1"
data="com.vuldroid.applcation"
Events injected: 1
```

Force-stop the Application

```
Shell
adb shell am force-stop <TARGET_PACKAGE>
```

Example:

```
Shell
adb shell am force-stop com.vulldroid.application
```

Use this before retesting a feature to begin from a clean app process state.

5.8 Application Storage Analysis

The Android application **sandbox** commonly contains:

- shared_prefs/
- databases/
- files/
- cache/
- code_cache/

Access depends on whether the app is **debuggable** or whether the course emulator supports **root access**.

5.8.1 Check run-as Access

Run:

```
Shell
adb shell run-as <TARGET_PACKAGE> id
```

Example:

```
Shell
adb shell run-as com.vulldroid.application id
```

Possible successful **output**:

None

```
uid=10xxx(u0_aXXX) gid=10xxx(u0_aXXX) groups=...
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation: $ adb shell run-as com.vulldroid.application.id
uid=10229(u0_a229) gid=10229(u0_a229) groups=10229(u0_a229),1004(input),1007(log),1011(adb),1015(sdcard_rw),1028(sdcard_r),1078(ext_data_rw),1079(ext_
obb_rw),3001(net_bt_admin),3002(net_bt),3003(inet),3006(net_bw_stats),3009(readproc),3011(uhid),3012(readtraces),50229(all_a229) context=u:r:runas_ap
p:s0:c229,c256,c512,c768
cdac@cdac-HP-Z2-Tower-G5-Workstation: ~$
```

```
Windows PowerShell
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell run-as owasp.sat.ag
oat id
uid=10230(u0_a230) gid=10230(u0_a230) groups=10230(u0_a230),1004(input),1007(log),1011(adb),1015(sdcard_rw),
1020(sdcard_r),1078(ext_data_rw),1079(ext_obb_rw),3001(net_bt_admin),3002(net_bt),3003(inet),3006(net_bw_s
tats),3009(readproc),3011(uhid),3012(readtraces),50230(all_a230) context=u:r:runas_app:s0:c512,c768
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> |
```

This means the application allows `run-as` access in the test environment.

Possible failure output:

None

```
run-as: package not debuggable: <TARGET_PACKAGE>
```

This means the application is **not debuggable**, so `run-as` cannot be used.

5.8.2 List Application Sandbox Files

If `run-as` works, run:

Shell

```
adb shell run-as <TARGET_PACKAGE> sh -c 'pwd; ls -lah; find .
-maxdepth 3 -type f -ls'
```

Expected **output** may show:

None

```
.
./shared_prefs/<file>.xml
./databases/<database>.db
./files/<file>
./cache/<file>
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ adb shell run-as com.vulndroid.application sh -c 'pwd; ls -lah; find . -maxdepth 3 -type f -ls'
/data/user/0/com.vulndroid.application
total 80
drwxr-xr-x 29 root root 4.0K 2026-08-12 11:18 .
drwxr-xr-x 29 root root 4.0K 2026-08-12 11:18 ..
lrwxr--r-- 1 root root 38 2026-08-12 11:13 adb_keys -> /product/etc/security/adb_keys
drwxr-xr-x 188 root root 2.0K 2026-09-17 11:17 apex
lrwxr--r-- 1 root root 11 2026-08-12 11:13 bin -> /system/bin
drwxr-xr-x 2 root root 4.0K 2026-08-12 11:13 bootstrap-apex
lrwxr--r-- 1 root root 58 2026-08-12 11:13 bugreports -> /data/user_de/0/com.android.shell/files/bugreports
drwxrwx--- 2 system cache 4.0K 2026-08-12 11:13 cache
drwxr-xr-x 3 root root 0 2026-09-17 11:17 config
lrwxr--r-- 1 root root 17 2026-08-12 11:13 d -> /sys/kernel/debug
drwxrwx--x 51 system system 4.0K 2026-09-17 11:18 data
d????????? ? ? ? ? ? data_mirror
drwxr-xr-x 2 root root 4.0K 2026-08-12 11:13 debug_ramdisk
drwxr-xr-x 19 root root 2.9K 2026-09-17 11:18 dev
lrwxr--r-- 1 root root 11 2026-08-12 11:13 etc -> /system/etc
l????????? ? ? ? ? ? init -> ?
-rw-r--r-- 1 ? ? ? ? ? init.environ.rc
```

```
Windows PowerShell
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell run-as owasp.sat.agoat sh -c 'pwd; ls -lah; find . -maxdepth 3 -type f -ls'
/data/user/0/owasp.sat.agoat
total 80
drwxr-xr-x 29 root root 4.0K 2026-09-09 12:16 .
drwxr-xr-x 29 root root 4.0K 2026-09-09 12:16 ..
lrwxr--r-- 1 root root 38 2026-09-09 12:06 adb_keys -> /product/etc/security/adb_keys
drwxr-xr-x 188 root root 2.1K 2026-09-28 17:25 apex
lrwxr--r-- 1 root root 11 2026-09-09 12:06 bin -> /system/bin
drwxr-xr-x 2 root root 4.0K 2026-09-09 12:06 bootstrap-apex
lrwxr--r-- 1 root root 58 2026-09-09 12:06 bugreports -> /data/user_de/0/com.android.shell/files
/bugreports
drwxrwx--- 2 system cache 4.0K 2026-09-09 12:06 cache
drwxr-xr-x 3 root root 0 2026-09-28 17:25 config
lrwxr--r-- 1 root root 17 2026-09-09 12:06 d -> /sys/kernel/debug
drwxrwx--x 51 system system 4.0K 2026-09-28 17:28 data
d????????? ? ? ? ? ? data_mirror
drwxr-xr-x 2 root root 4.0K 2026-09-09 12:06 debug_ramdisk
drwxr-xr-x 18 root root 3.0K 2026-09-28 17:26 dev
lrwxr--r-- 1 root root 11 2026-09-09 12:06 etc -> /system/etc
l????????? ? ? ? ? ? init -> ?
-rw-r--r-- 1 ? ? ? ? ? init.environ.rc
d????????? ? ? ? ? ? linkerconfig
drwx----- 2 root root 16K 2026-09-09 12:16 lost+found
d????????? ? ? ? ? ? metadata
drwxr-xr-x 17 root system 368 2026-09-28 17:25 mnt
drwxr-xr-x 2 root root 4.0K 2026-09-09 12:06 odm
drwxr-xr-x 2 root root 4.0K 2026-09-09 12:06 odm_dkms
drwxr-xr-x 2 root root 4.0K 2026-09-09 12:06 oem
d????????? ? ? ? ? ? postinstall
dr-xr-xr-x 398 root root 0 2026-09-28 17:25 proc
drwxr-xr-x 11 root root 4.0K 2026-09-09 12:16 product
lrwxr--r-- 1 root root 21 2026-09-09 12:06 sdcard -> /storage/self/primary
drwxr-xr-x 2 root root 4.0K 2026-09-09 12:06 second_stage_resources
drwx---x--- 5 shell everybody 188 2026-09-28 17:28 storage
dr-xr-xr-x 13 root root 0 2026-09-28 17:25 sys
drwxr-xr-x 13 root root 4.0K 2026-09-09 12:16 system
```

5.8.3 Root-Capable Emulator Method

If the emulator allows root access, run:

```
Shell
adb shell
su
```

Then access the authorized target application directory:

```
Shell
cd /data/data/<TARGET_PACKAGE>
ls -lah
```

Example:

```
Shell
cd /data/data/com.vulndroid.application
ls -lah
```

Use this method only on the authorized lab emulator.

```
Windows PowerShell
PS C:\Users\winay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell
emu64xal6k:/ $ su
emu64xal6k:/ # ls -lah
total 160
drwxr-xr-x 29 root root 4.0K 2026-09-09 12:16 .
drwxr-xr-x 29 root root 4.0K 2026-09-09 12:16 ..
lrw-r--r-- 1 root root 30 2026-09-09 12:06 adb_keys -> /product/etc/security/adb_keys
drwxr-xr-x 188 root root 2.1K 2026-09-20 17:25 apex
lrw-r--r-- 1 root root 11 2026-09-09 12:06 bin -> /system/bin
drwxr-xr-x 2 root root 4.0K 2026-09-09 12:06 bootstrap-apex
lrw-r--r-- 1 root root 50 2026-09-09 12:06 bugreports -> /data/user_de/0/com.android.shell/files
/bugreports
drwxrwx--- 2 system cache 4.0K 2026-09-09 12:06 cache
drwxr-xr-x 3 root root 0 2026-09-20 17:25 config
lrw-r--r-- 1 root root 17 2026-09-09 12:06 d -> /sys/kernel/debug
drwxrwx--- 51 system system 4.0K 2026-09-20 17:28 data
drwx----- 9 root system 180 2026-09-20 17:25 data_mirror
drwxr-xr-x 2 root root 4.0K 2026-09-09 12:06 debug_randisk
drwxr-xr-x 18 root root 3.0K 2026-09-20 17:26 dev
lrw-r--r-- 1 root root 11 2026-09-09 12:06 etc -> /system/etc
lrwxr-x--- 1 root shell 16 2026-09-09 12:06 init -> /system/bin/init
-rwxr-x--- 1 root shell 621 2026-09-09 12:06 init.environ.rc
drwxr-xr-x 31 root root 660 2026-09-20 17:28 libexec-bin
```

5.9 Send a Known Intent to an Authorized Component

Use this section only after identifying the activity, intent action, URI scheme, and expected parameters from static analysis.

Start a Known Activity

Shell

```
adb shell am start -W -n  
<TARGET_PACKAGE>/<FULLY_QUALIFIED_ACTIVITY>
```

Example format:

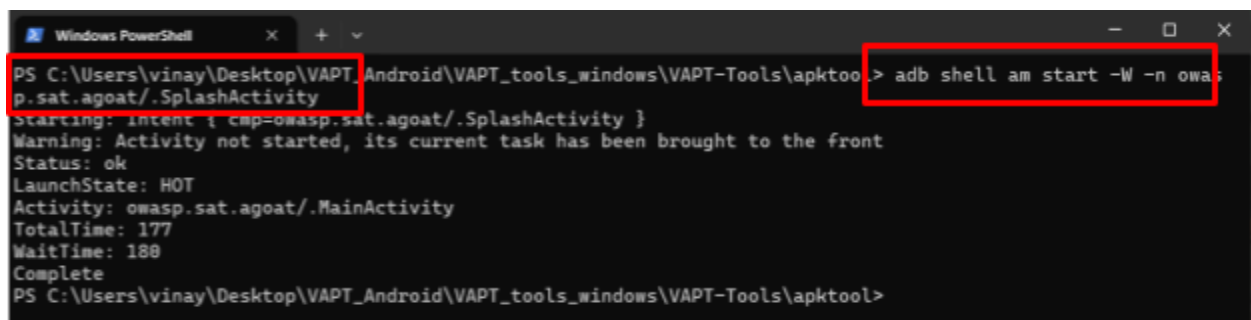
Shell

```
adb shell am start -W -n owasp.sat./.SplashActivity
```

Expected output resembles:

None

```
Starting: Intent { cmp=com.vulldroid.application/.SplashActivity }  
Status: ok  
Activity: com.vulldroid.application/.MainActivity
```



```
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell am start -W -n owasp.sat.agoat/.SplashActivity  
Starting: Intent { cmp=owasp.sat.agoat/.SplashActivity }  
Warning: Activity not started, its current task has been brought to the front  
Status: ok  
LaunchState: HOT  
Activity: owasp.sat.agoat/.MainActivity  
TotalTime: 177  
WaitTime: 180  
Complete  
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool>
```

6. ADB Logcat Analysis for Windows and Linux

Keep Logcat analysis separate from general ADB commands. Logcat is used to capture application and Android system logs while you perform the assigned test workflow.

6.1 Clear Existing Logcat Data

Before starting a new test, clear old emulator logs:

```
Shell  
adb logcat -c
```

This helps ensure that later output belongs to the current lab activity.

6.2 Start Full Logcat Capture

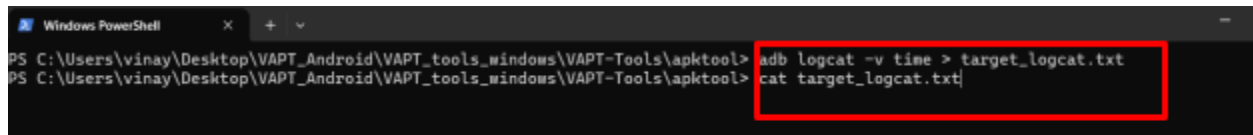
Run:

```
Shell  
adb logcat -v time > target_logcat.txt
```

Expected result: Logcat continuously writes timestamped logs to **target_logcat.txt**.

While the command is running:

1. Open the **target application**.
2. Perform the assigned module/workflow.
3. Reproduce the suspected behavior.
4. Return to the terminal.
5. Press **Ctrl + C** to stop the capture.



The screenshot shows a Windows PowerShell terminal window with the following commands and output:

```
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb logcat -v time > target_logcat.txt  
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> cat target_logcat.txt
```

The second command and its output are highlighted with a red rectangle.

6.3 Capture Logs for One Application Process

First, open the target app and obtain its PID:

```
Shell  
adb shell pidof <TARGET_PACKAGE>
```

Example output:

```
None  
12345
```

Then run:

```
Shell  
adb logcat --pid=<PID> -v time
```

Example:

```
Shell  
adb logcat --pid=12345 -v time
```

This displays logs only from the target application process.
To save the process-specific logs:

```
Shell  
adb logcat --pid=<PID> -v time > target_app_logcat.txt
```

```

PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb shell pidof owasp.sat.agoa
4904
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb logcat --pid=4904
----- beginning of main
09-20 18:25:43.842 4904 4904 I owasp.sat.agoa: hiddenapi: Accessing hidden method Ldalvik/system/CloseGuard;
->get()Ldalvik/system/CloseGuard; (runtime_flags=CorePlatformApi, domain=core-platform, api=unsupported,core-pla
tform-api) from Lokhttp3/internal/platform/AndroidPlatform$CloseGuard; (domain=app, TargetSdkVersion=
26) using reflection: allowed
09-20 18:25:43.843 4904 4904 I owasp.sat.agoa: hiddenapi: Accessing hidden method Ldalvik/system/CloseGuard;
->open(Ljava/lang/String;)V (runtime_flags=CorePlatformApi, domain=core-platform, api=unsupported,core-pl
atform-api) from Lokhttp3/internal/platform/AndroidPlatform$CloseGuard; (domain=app, TargetSdkVersion=26) us

```

6.4 Filter Logcat Using the Package Name

Linux:

Shell

```
adb logcat -v time | grep -i <TARGET_PACKAGE>
```

Example:

Shell

```
adb logcat -v time | grep -i com.vuldroid.application
```

Windows PowerShell:

Shell

```
adb logcat -v time | Select-String -Pattern "<TARGET_PACKAGE>"
```

Example:

Shell

```
adb logcat -v time | Select-String -Pattern
"com.vuldroid.application"
```

Package-name filtering may not capture every app log because Android log lines often use a developer-defined tag instead of the package name. PID filtering is normally more accurate after the app is running.

```
Windows PowerShell
C:\Users\...ay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb logcat -v time | Select-String -Pattern *owasp sat.agoat*

09-28 18:25:43.658 V/ActivityTaskManager( 772): TaskLaunchParamsModifier: phase=2 task=Task{fa7fc9a #11 type=standard A=18230:owasp.sat.agoat} activity=ActivityRecord{41888653 u0 owasp.sat.agoat/.TrafficActivity t-1}
display-area-from-source=DefaultTaskDisplayArea@182802896 display-id=0 task-display-area-windowing-mode=1
suggested-display-area=DefaultTaskDisplayArea@182802896 inherit-from-source=fullscreen non-freeform-task-display-area
display-area=DefaultTaskDisplayArea@182802896 skip-bounds=fullscreen
09-28 18:25:43.658 I/ActivityTaskManager( 772): START u0 {xflg=0x4 cmp=owasp.sat.agoat/.TrafficActivity} with LAUNCH_MULTIPLE
from uid 18230 (owasp.sat.agoat) (sr=115979026) (BAL_ALLOW_VISIBLE_WINDOW) result code=0
09-28 18:25:43.660 V/WindowManagerShell( 2288): Transition requested (#46): android.os.BinderProxy@f75c09a
TransitionRequestInfo { type = OPEN, triggerTask = TaskInfo{userId=0 taskId=11 effectiveUid=18230 seqId=0 displayId=0
isRunning=true baseIntent=Intent { flg=0x18000000 cmp=owasp.sat.agoat/.SplashActivity }
baseActivity=ComponentInfo{owasp.sat.agoat/owasp.sat.agoat.MainActivity}
```

6.5 Filter Logs by Severity

Android log severity levels include:

Show only errors:

```
Shell
adb logcat *:E
```

Show warnings and errors:

```
Shell
adb logcat *:W
```

Show a specific application tag if identified during testing:

```
Shell
adb logcat -s <LOG_TAG>:V
```

Example format:

```
Shell
adb logcat -s VulDroid:V
```

6.6 Search Saved Logs for Sensitive Information

After saving a Logcat file, search it for important security indicators.

Linux:

```
Shell
grep -Ein
'password|passwd|token|secret|authorization|bearer|cookie|session|a
pi_key|apikey|http://|https://|exception|sqlite|database'
target_logcat.txt
```

Windows PowerShell:

```
Shell
Select-String -Path .\target_logcat.txt -Pattern
"password|passwd|token|secret|authorization|bearer|cookie|session|a
pi_key|apikey|http://|https://|exception|sqlite|database"
```

Check whether the app logs:

- Username, password, PIN, OTP, token, API key, or session value
- Authorization
- Personal information
- Full server requests or responses
- Database queries or records
- Internal IP addresses, URLs, file paths, or endpoints
- Stack traces and detailed exception messages
- Root/debug/security-check results

```
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> sqlite
```

```
target_logcat.txt:9619:09-20 18:26:03.831 V/ActivityTaskManager( 772): TaskLaunchParamsModifier: phase=2 task=Task{fa7fc9a #11 type=standard
A=18230:owasp.sat.agoat} activity=ActivityRecord{286155894 u0 owasp.sat.agoat/.InsecureStorageSQLiteActivity t-1}
display-area-from-source=DefaultTaskDisplayArea@182802896 display-id=8 task-display-area-windowing-mode=1
suggested-display-area=DefaultTaskDisplayArea@182802896 inherit-from-source=fullscreen non-freeform-task-display-area
display-area=DefaultTaskDisplayArea@182802896 skip-bounds-fullscreen
target_logcat.txt:9620:09-20 18:26:03.832 I/ActivityTaskManager( 772): START u0 {xflg=0x4 cmp=owasp.sat.agoat/.InsecureStorageSQLiteActivity}
with LAUNCH_MULTIPLE from uid 18230 (owasp.sat.agoat) (sr=1319664) (BAL_ALLOW_VISIBLE_WINDOW) result code=0
target_logcat.txt:9626:09-20 18:26:03.882 V/WindowManagerShell( 2288): Transition requested (#49): android.os.BinderProxy@5cb61d9
TransitionRequestInfo { type = OPEN, triggerTask = TaskInfo{userId=0 taskId=11 effectiveUid=18230 seqId=0 displayId=0 isRunning=true
baseIntent=Intent { flg=0x10000000 cmp=owasp.sat.agoat/.SplashActivity }
baseActivity=ComponentInfo{owasp.sat.agoat/owasp.sat.agoat.SplashActivity}
topActivity=ComponentInfo{owasp.sat.agoat/owasp.sat.agoat.InsecureStorageSQLiteActivity} origActivity=null
realActivity=ComponentInfo{owasp.sat.agoat/owasp.sat.agoat.SplashActivity} realActivityIsAppLockEnabled=false numActivities=3
false capturedLink=null capturedLinkTimestamp=0 requestedVisibleTypes=503 topActivityRequestOpenInBrowserEducationTimestamp=0
...}
```

```
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\apktool> adb logcat -d | Select-String -Pattern *FATAL EXCEPTION|AndroidRun
time|owasp.sat.agoat"
```

```
09-20 18:25:43.658 772 3568 V ActivityTaskManager: TaskLaunchParamsModifier: phase=2 task=Task{fa7fc9a #11 type=standard
A=18230:owasp.sat.agoat} activity=ActivityRecord{41880653 u0 owasp.sat.agoat/.TrafficActivity t-1}
display-area-from-source=DefaultTaskDisplayArea@182802896 display-id=8 task-display-area-windowing-mode=1
suggested-display-area=DefaultTaskDisplayArea@182802896 inherit-from-source=fullscreen non-freeform-task-display-area
display-area=DefaultTaskDisplayArea@182802896 skip-bounds-fullscreen
09-20 18:25:43.658 772 3568 I ActivityTaskManager: START u0 {xflg=0x4 cmp=owasp.sat.agoat/.TrafficActivity} with LAUNCH_MULTIPLE from uid
18230 (owasp.sat.agoat) (sr=115979826) (BAL_ALLOW_VISIBLE_WINDOW) result code=0
09-20 18:25:43.660 2288 2381 V WindowManagerShell: Transition requested (#46): android.os.BinderProxy@ff75c99a TransitionRequestInfo { type =
OPEN, triggerTask = TaskInfo{userId=0 taskId=11 effectiveUid=18230 seqId=0 displayId=0 isRunning=true baseIntent=Intent { flg=0x10000000
cmp=owasp.sat.agoat/.SplashActivity } baseActivity=ComponentInfo{owasp.sat.agoat/owasp.sat.agoat.MainActivity}
topActivity=ComponentInfo{owasp.sat.agoat/owasp.sat.agoat.TrafficActivity} origActivity=null
realActivity=ComponentInfo{owasp.sat.agoat/owasp.sat.agoat.SplashActivity} realActivityIsAppLockEnabled=false numActivities=2
lastActiveTime=3682693 supportsMultiWindow=true supportsMultiWindowWithoutConstraints=true resizeMode=1 isResizable=true minWidth=1
minHeight=1 defaultMinSize=228 token=WCT{android.os.BinderProxy@b84d4aa} topActivityType=1 pictureInPictureParams=null
shouldDockBigOverlays=false launchIntoPictureHostTaskId=-1 lastParentTaskIdBeforePip=-1 displayCutoutSafeInsets=Rect(0, 0 - 0, 0)
topActivityInfo=ActivityInfo{6eeldcb owasp.sat.agoat.TrafficActivity} launchCookies=[] positionInParent=Point(0, 0) parentTaskId=-1
isFocused=true isInteractive=true isVisible=true isVisibleRequested=true isTopActivityNoDisplay=false isSleeping=false locusId=null
displayAreaFeatureId=1 isTopActivityTransparent=false isActivityStackTransparent=false lastNonFullscreenBounds=null leafTaskBoundsFromOptions=
false capturedLink=null capturedLinkTimestamp=0 requestedVisibleTypes=503 topActivityRequestOpenInBrowserEducationTimestamp=0
...}
```

7. Frida analysis

7.1 Verify Emulator and ADB Connection

1. Start the configured Android emulator from Android Studio Device Manager.
2. Wait for the Android home screen to load completely.
3. Open PowerShell (Windows) or Terminal (Ubuntu).
4. Run:

```
Shell
adb devices
```

Expected result:

```
None
List of devices attached
emulator-5554    device
```

If the output shows **offline**, wait for the emulator to complete **booting**. If necessary, restart the **ADB** server:

```
Shell
adb kill-server
adb start-server
adb devices
```

7.2 Identify Emulator Architecture

Before downloading Frida server, determine the Android emulator CPU ABI.

Run:

```
Shell
adb shell getprop ro.product.cpu.abi
```

The lab emulator output is:

```
None
x86_64
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ adb shell getprop ro.product.cpu.abi
x86_64
```

Optionally display all supported ABIs:

```
Shell
adb shell getprop ro.product.cpu.abi
```

Frida-server Selection Table

ro.product.cpu.abi Output	Frida Server Architecture to Download
x86_64	android-x86_64
x86	android-x86
arm64-v8a	android-arm64
armeabi-v7a	android-arm

For this lab emulator, the correct architecture is:

None

android-x86_64

7.3 Download and Extract Matching Frida Server

7.3.1 Open the Official Frida Releases Page

Open the Frida releases page in a browser:

<https://github.com/frida/frida/releases>

7.3.2 Select the Exact Matching Server File

The installed client version is:

None

17.18.0

The emulator architecture is:

None

x86_64

Therefore download exactly:

None

`frida-server-17.18.0-android-x86_64.xz`

Do not download these files for the lab emulator:

None

`frida-server-17.18.0-android-arm.xz`

`frida-server-17.18.0-android-arm64.xz`

`frida-server-17.18.0-android-x86.xz`

`frida-server-17.18.0-linux-arm64-musl.xz`

 frida-server-17.18.0-android-arm.xz	sha25...		7.89 MB	last week
 frida-server-17.18.0-android-arm64.xz	sha25...		17.1 MB	last week
 frida-server-17.18.0-android-x86.xz	sha25...		16.8 MB	last week
 frida-server-17.18.0-android-x86_64.xz	sha25...		32.1 MB	last week
 frida-server-17.18.0-linux-arm64-musl.xz	sha25...		9.29 MB	last week

7.3.3 Extract the .xz Archive

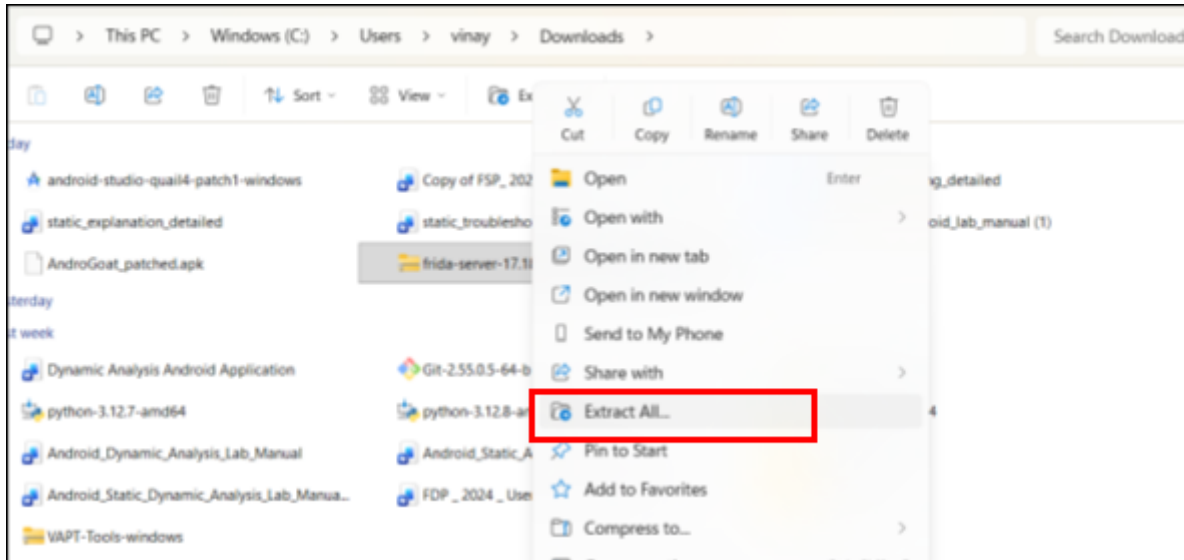
Go to the folder where the server archive was downloaded.

Example:

Shell

`cd /mnt/work/Downloads`

Extract the archive:



Or

Shell

```
unxz frida-server-17.18.0-android-x86_64.xz
```

After **extraction**, confirm that both files are visible:

Shell

```
ls -lh frida-server-17.18.0-android-x86_64*
```

Expected files:

None

```
frida-server-17.18.0-android-x86_64
frida-server-17.18.0-android-x86_64.xz
```

The extracted server binary in the lab is approximately 116 MB, while the compressed .xz file is approximately 33.7 MB.

Name	Size	Location
 frida-server-17.18.0-android-x86_64	116.0 MB	
 frida-server-17.18.0-android-x86_64.xz	33.7 MB	

7.4 Push and Start Frida Server

7.4.1 Push Frida Server to the Emulator

From the folder containing the extracted binary, run:

Shell

```
adb push frida-server-17.18.0-android-x86_64 /data/local/tmp
```

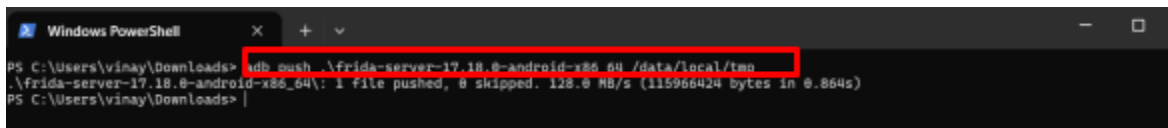
Expected result resembles:

None

```
frida-server-17.18.0-android-x86_64: 1 file pushed
```



```
cdac@cdac-HP-72-Tower-G5-Workstation: /mnt/work/Downloads$ adb push frida-server-17.18.0-android-x86_64 /data/local/tmp
frida-server-17.18.0-android-x86_64: 1 file pushed, 0 skipped. 87.6 MB/s (115966424 bytes in 1.263s)
```



```
PS C:\Users\vinay\Downloads> adb push .\frida-server-17.18.0-android-x86_64 /data/local/tmp
.\frida-server-17.18.0-android-x86_64: 1 file pushed, 0 skipped. 128.0 MB/s (115966424 bytes in 0.864s)
PS C:\Users\vinay\Downloads>
```

7.4.2 Open an ADB Shell and Obtain Root Access

Run:

Shell

```
adb shell
su
```

A root-capable Android emulator prompt changes from a normal shell prompt to a root prompt, commonly ending with #.

Example:

None

```
emu64x86k:/ $ su
emu64x86k:/ #
```

7.4.3 Change to the Temporary Directory

Inside the emulator shell, run:

Shell

```
cd /data/local/tmp
```

7.4.4 Give Execute Permission to Frida Server

Run:

Shell

```
chmod +x frida-server-17.18.0-android-x86_64
```

The screenshot shows a Windows PowerShell terminal window with two tabs. The active tab shows the following commands and output:

```
PS C:\Users\vinay> adb shell
emu64x86k:/ $ su
emu64x86k:/ # cd data/local/tmp
emu64x86k:/data/local/tmp # ls
frida-server-17.18.0-android-x86_64
emu64x86k:/data/local/tmp # chmod +x frida-server-17.18.0-android-x86_64
emu64x86k:/data/local/tmp # ./frida-server-17.18.0-android-x86_64 &
[1] 23685
emu64x86k:/data/local/tmp # |
```

Red boxes highlight the `adb shell` command, the `ls` command output, and the `./frida-server-17.18.0-android-x86_64 &` command.

7.4.5 Start Frida Server in the Background

Run:

Shell

```
./frida-server-17.18.0-android-x86_64 &
```

Add `&` at the end of the command to keep Frida server running in the background, so you can continue using the same terminal.

The lab session returned a background job and process ID similar to:

None

[1] 3691

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads$ adb shell
emu64xa16k:/ $ su
emu64xa16k:/ # cd data/local/tmp
emu64xa16k:/data/local/tmp # chmod +x frida-server-17.18.0-android-x86_64
emu64xa16k:/data/local/tmp # ./frida-server-17.18.0-android-x86_64 &
[1] 3691
emu64xa16k:/data/local/tmp #
```

7.4.6 Confirm that Frida Server is Running

Open a new workstation terminal, or exit the emulator shell, then run:

Shell

```
adb shell ps -A | grep frida
```

On Windows PowerShell use:

Shell

```
adb shell ps -A | Select-String frida
```

Expected result: A running process containing the Frida server name.

If no server process appears, inspect the emulator shell for errors and confirm the server architecture/version again.

```
PS C:\Users\vinay> adb shell ps -A | Select-String frida
root      23685 23348   13418036  44272 0      0 S frida-server-17.18.0-android-x86_64

PS C:\Users\vinay> frida-ps -U
PID Name
-----
5330 .adservices
4993 Camera
5608 Chrome
4274 Files
3112 Google
4991 Phone
4053 Photos
2423 SIM Toolkit
2483 Settings
12230 adb
572 adbd
494 android.hardware.audio.service
516 android.hardware.authsecret-service.example
2108 android.hardware.biometrics.fingerprint-service.ranchu
510 android.hardware.bluetooth-service.default
```

7.5 Verify Frida Connection

On the workstation, run:

```
Shell  
frida-ps -U
```

The `-U` option targets the ADB-connected Android emulator/device.
Expected result: Frida displays a list of running Android processes.

7.6 Spawn the Target Application with Frida

7.6.1 Identify the Target Package Name

The evidence uses VulDroid as the target application. Its package name was identified with:

```
Shell  
adb shell pm list packages | grep vuldroid
```

Output:

```
None  
package:com.vuldroid.application
```

On Windows **PowerShell**, use:

```
Shell  
adb shell pm list packages | Select-String vuldroid
```

7.6.2 Spawn the Target Application

Run:

```
Shell
frida -U -f com.vuldroid.application
```

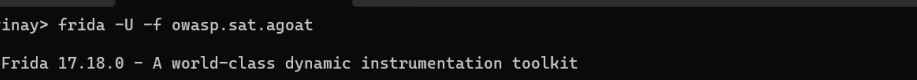
Meaning:

Option / Value	Meaning
frida	Starts the Frida command-line client
-U	Uses the ADB-connected Android emulator/device
-f	Spawns the target application instead of only attaching to an already-running process
com.vuldroid.application	Authorized target application package name

Expected **output** includes messages similar to:

```
None
Connected to Android Emulator 5554 (id=emulator-5554)
Spawned `com.vuldroid.application`. Resuming main thread!
[Android Emulator 5554::com.vuldroid.application ]->
```

This prompt confirms that **Frida** successfully connected to the **emulator** and **spawned** the target **application**.



```

PS C:\Users\vinay> frida -U -f owasp.sat.agoat

  /---\
  | C |   Frida 17.18.0 - A world-class dynamic instrumentation toolkit
  >
  /- / |   Commands:
  . . .     help      -> Displays the help system
  . . .     object?   -> Display information about 'object'
  . . .     exit/quit -> Exit
  . . .
  . . .     Prefer a GUI? Luma is the official Frida app, with a live REPL,
  . . .     persistent sessions & collaboration. https://luma.frida.re/
  . . .
  . . .     Connected to Android Emulator 5554 (id=emulator-5554)
Spawned 'owasp.sat.agoat'. Resuming main thread!
[Android Emulator 5554:owasp.sat.agoat ]-> |
  
```

Linux:

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads$ adb shell pm list packages|grep vuldroid
package:com.vuldroid.application
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads$ frida -U -f com.vuldroid.application
```

```
/_/_/ | Frida 17.18.0 - A world-class dynamic instrumentation toolkit
|(__|_|
|>_|_|
|_|_|
|_|_|
./_|_|_| help      -> Displays the help system
. . . . object?    -> Display information about 'object'
. . . . exit/quit -> Exit
. . . .
. . . . Prefer a GUI? Luma is the official Frida app, with a live REPL,
. . . . persistent sessions & collaboration. https://luma.frida.re/
. . . .
. . . . Connected to Android Emulator 5554 (id=emulator-5554)

Spawned `com.vuldroid.application`. Resuming main thread!
[Android Emulator 5554::com.vuldroid.application ]->
```

7.6.3 Exit the Frida Console

At the Frida interactive prompt, type `exit` or press **Ctrl + D**.

7.7 Basic Frida Script Test

After confirming the **setup**, create a basic **script** that verifies Java runtime access without printing sensitive data.

Create `frida test.js`:

Shell

```
Set-Content -Path .\frida_test.js -Value @( 'setTimeout(function ()
{', ' if (Java.available) {', ' Java.perform(function () {', ' try
```

```
{', ' console.log("[+] Frida attached successfully");', '
console.log("[+] Java runtime is available");', ' } catch (error)
{', ' console.log("[-] Error inside Java.perform");', '
console.error(error);', ' }', ' }));', ' } else {', '
console.log("[-] Java is not available in this process");', ' }',
'}, 500);' )
    console.log("[+] Java runtime is available");
});
```

Run it against the authorized target **application**:

Shell

```
frida -U -f com.vulndroid.application -l frida_test.js
```

Expected output:

None

```
[+] Frida attached successfully
[+] Java runtime is available
```

Use this test before attempting any class-specific observation script.

```
PS C:\Users\vinay\Downloads> frida -U -f owasp.sat.agoat -l frida_test.js

Frida 17.18.0 - A world-class dynamic instrumentation toolkit

Commands:
  help      -> Displays the help system
  object?   -> Display information about 'object'
  exit/quit -> Exit

Prefer a GUI? Luma is the official Frida app, with a live REPL,
persistent sessions & collaboration. https://luma.frida.re/

Connected to Android Emulator 5554 (id=emulator-5554)
Spawning 'owasp.sat.agoat'. Resuming main thread!
[Android Emulator 5554:owasp.sat.agoat ]-> [+] Frida attached successfully
[+] Java runtime is available
[Android Emulator 5554:owasp.sat.agoat ]->
[Android Emulator 5554:owasp.sat.agoat ]-> |
```

8. Identify Security Findings and Prepare Observations

Steps:

1. Select another APK provided or an APK from an external source for analysis.
2. Perform the **dynamic analysis** using the tools and techniques covered in this lab, including **ADB-based analysis, application storage analysis, Intent testing, Logcat analysis, runtime application observation, and Frida-based analysis**.
3. Perform the assigned application workflows and identify any **security-relevant behavior or weaknesses** observed during runtime analysis.
4. For each finding, prepare an observation containing:
 - **Finding/Issue**
 - **Location/Component** (activity, service, receiver, provider, storage location, runtime behavior, etc.)
 - **Steps/Command Used** to reproduce the observation
 - **Evidence** such as ADB output, Logcat output, Frida output, or screenshot
 - **Security impact**
 - **Observation/Conclusion**
5. Prepare a brief **dynamic security analysis report** summarizing the findings identified in the selected APK, along with the supporting evidence collected during the practical.